

Troubleshooting Issues in Linux Applications

In order to address issues in applications, the author has listed what needs to be done whenever you install a third party application on RHEL systems. The steps are almost the same for other Linux flavours.

Before going any further, let's take a look at certain aspects of applications:

- Shared libraries
- How linking shared libraries with applications works
- How applications really find shared libraries
- Real-time troubleshooting relating to application issues.

For the purposes of this article, I have used RHEL (Red Hat Enterprise Linux) as the base OS. However, the steps are almost the same for other Linux flavours.

Most Linux applications are dynamically linked with the shared libraries they use. Shared libraries are built in a way that allows their functions to be mapped into an application's memory when the application is executed. Since the required code is provided by the library at runtime, it is not copied to the application itself.

Linking shared libraries with the application gives the following benefits:

- The application binary executables are smaller, since the library code is not copied into them.
- The library can be shared with multiple, simultaneously executing programs. This takes up less system memory.
- Shared libraries make it easier to fix bugs in the library code. Most fixes to a shared library do not require every application linked with it to be rebuilt.

Linking with shared libraries

When an application is built, it is linked with the shared libraries that provide the functions that it uses. The compiler checks the libraries for the required symbols and confirms they exist.

Identifying information about each required library is embedded in the executable. This can be the absolute path name of the library but it is more commonly the *DT_SONAME* field of the shared library. The *DT_SONAME* includes the name and version of the shared library, so that the application will use the correct version at runtime. This field is set when the shared library is created with the *-soname* option to the *ln* linker. The *objdump* command includes the field when it displays information about the shared library. You can view the information about *libpthread.so.0* in Figure 1.

